Moodle の拡張機能を作ってみよう！

2 日間ハンズオン・参加者用手順書

最新版スライド「moodle(1).pdf」（全 107 ページ）と対応

スクリーンの説明を見る → 自分で操作する → この資料で確認 → 困ったら GitHub で復帰

使い方

☐ 講師が映しているスライドの「ページ番号」を確認します。
☐ この資料の同じページ範囲を開き、「操作」と「成功条件」を確認します。
☐ 迷ったら GitHub の docs/slide-index.md から対応する完成コードを探します。
☐ 続きは次の開催日へ。日付ではなく、最後に進んだスライド番号をメモしておきます。
※ この拡張機能は大学の正規ログイン画面のクリックだけを対象とし、パスワードや MFA コードは扱いません。実サイト認証の通し動作は未検証です。

1. スライドと作業の早見表

部・ページ	参加者がやること	GitHub の復帰ポイント
導入 1–5	Chrome・VS Code を準備	docs/participant-guide.md
第 1 部 6–21	manifest.json／content.js を作成	code/01-p020-first-extension
第 2 部 22–33	Console でボタンを探してクリック	snippets/（p027, p029, p031）
第 3 部 34–46	自動クリック・URL 判定	code/03-p036… ／ 03-p043…
第 4 部 47–60	2 画面目も自動クリック	code/04-p055-two-buttons
第 5 部 61–79	再実行を抑える	code/05-p073-retry-guard
第 6 部 80–107	ポップアップ／オン・オフ	code/06-p093… ／ 06-p102…

2. まずは作業環境をつくる（p4、p9〜20）

- Chrome と VS Code をインストールしておきます。
- Windows のコマンドプロンプトで、次を 1 行ずつ実行します。

```
mkdir wakayama-moodle-auto-login
cd wakayama-moodle-auto-login
code .
```

☐ p12：manifest.json を作る。
☐ p16：content.js を作る。

☐ p18〜19：Chrome のアドレス欄に chrome://extensions/ を入れる。デベロッパーモード → 「パッケージ化されていない拡張機能を読み込む」→ 作成したフォルダーを選択。

☐ p20：Moodle を開いて F12 → Console → Ctrl+R でメッセージを確認。

確認 ☐ p20 「Moodle 拡張機能が動作しました！」と URL が表示された

完成コード：code/01-p020-first-extension/　（manifest.json と content.js の 2 ファイル）

3. HTML からログインボタンを探す（p22〜33）

☐ p26：Moodle が未ログインのときに「ログイン」を右クリック →「検証」を選択。

☐ p27：F12 → Console でログインリンクを探すコードを実行。

```
const loginButton = document.querySelector(
  'a[href$="/login/index.php"]'
);
console.log(loginButton);
```

☐ p29：同じ Console で loginButton.click(); を実行し、画面遷移を確認。

☐ p31：ボタンがなかった場合もエラーにならない if 文を試す。

確認 ☐ p29 ボタンをマウスで押さなくてもログイン画面に進んだ

この部では拡張機能のファイルは変更しません。演習用のコード：snippets/02-p027…、02-p029…、02-p031…

4. 自動クリックにする（p34〜46）

☐ p36：content.js に自動クリックのコードを書く（p036 の完成コードを参照）。

☐ p39：Ctrl+S → chrome://extensions/で「更新」 → Moodle も再読み込み。

☐ p42：Console で window.location.pathname を表示し、URL のパスを調べる。

☐ p43：/2026/ のトップページでだけクリックする if 条件を追加する。

確認 ☐ p43 トップページで自動クリックし、他ページでは不要な操作をしない

完成コード：code/03-p036-auto-click/ → code/03-p043-path-check/

5. 2 つのログイン画面に対応（p47〜60）

☐ p50：大学 ID ログイン画面の Console で大学 ID 用リンクを探す。

```
const universityButton = document.querySelector(
  'a[href$="/auth/oidc/"]'
);
console.log(universityButton);
```

☐ p51：universityButton.click(); を実行。

☐ p53：else if を使った分岐を試す。

☐ p55：content.js を 2 画面分の処理に書き換える。

確認 ☐ p58 大学 ID 認証に向かうところまで確認した

完成コード：code/04-p055-two-buttons/。注意：スライド p55 の 2 画面目パスは /2026/login/index_form.html ですが、最初のリンクは /2026/login/index.php へ進みます。実際のページと Console で違いを確かめてください。

6. 繰り返し操作を防ぐ（p61〜79）

- ☐ p65〜69：Console で sessionStorage.setItem/getItem/removeItem を試す。
- ☐ p73：content.js を「クリック済みの記録を残す」版に更新する。
- ☐ p78：実際に画面を再読み込みし、不要な繰り返しがないか調べる。

```
sessionStorage.removeItem("moodle-first-clicked");
sessionStorage.removeItem("moodle-second-clicked");
```

確認 ☐ p78 繰り返しを抑えられた。再テストする時は記録を消せる

完成コード：code/05-p073-retry-guard/。これは講習用の簡易な再実行防止で、認証成功を保証するものではありません。

7. ON／OFF 設定面を作る（p80〜107）

- ☐ p85：popup.html を新規作成（チェックボックスを含む画面）。
- ☐ p90：manifest.json に storage 権限と action.default_popup を追加。
- ☐ p93：popup.js を新規作成（設定を読み書きする）。
- ☐ p101〜102：content.js の先頭で設定の有効・無効を判定する。
- ☐ p105：拡張機能アイコンを開き、OFF で自動操作が止まり、ON で動くかテスト。

確認 ☐ p105 拡張機能の設定画面が開き、ON/OFF が反映された

完成コード：code/06-p093-popup-ready/ → code/06-p102-complete/

8. 困ったら最初に見ること

症状	確認すること
読み込み時エラー	manifest.json の場所、拡張子、カンマ／括弧
Console に何も出ない	拡張機能の更新、Moodle 側の再読み込み、対象 URL
querySelector が null	ログイン済みか、Elements の href、iframe 内か
2 画面目だけ動かない	実際の pathname、p55 と p73 の条件の違い
再試行できない	sessionStorage のクリック記録を削除
ON/OFF が効かない	p90 の storage 権限、popup.js、別拡張機能の重複

詳しい対応：GitHub の docs/troubleshooting.md。パスワード・認証コード・セッション情報は公開しないでください。

9. 2 日間の進メモ

1 日目　最後に進んだスライド：p _____　　完了した部：_____

2 日目　開始したスライド：p _____　　最後に進んだスライド：p _____

目安：1 日目は p46（第 3 部）を目標、早ければ p60。遅ければ p33 で終了しても問題ありません。2 日目は続きから開始します。

GitHub の URL（配布後に記入）：_____

一番大切なのは、全部暗記することではなく「困ったら調べて直せる」ようになることです。